



Ragad Karaman

Computer Engineer

CONTACT

+90 551 083 31 24

ragadkaraman22@gmail.com

<https://github.com/raghad-karaman>

<https://ragadkaraman.vercel.app>

SKILLS

- Programming Languages: C#, JavaScript, TypeScript, Python, PHP, C
- Web Technologies: HTML5, CSS3, React, Next.js, Tailwind CSS
- Backend: ASP.NET Core, ASP.NET MVC, Entity Framework, Laravel, FastAPI
- Mobile: React Native
- Database: PostgreSQL, MySQL, SQL Server
- Artificial Intelligence: TensorFlow, YOLO, OpenCV, Scikit-learn, XGBoost, Random Forest
- Tools: Git, GitHub, Postman, Playwright MCP

ABOUT ME

I graduated from the Department of Computer Engineering at Gaziantep Islamic Science and Technology University. I have carried out academic and applied work in software development, web technologies, database systems, artificial intelligence, and machine learning. In addition to programming languages such as C#, C, Python, PHP, and JavaScript, I can work with WordPress technology, and I have developed projects using HTML5, CSS3, Bootstrap, jQuery, React, Next.js, React Native, ASP.NET, Laravel, and FastAPI.

Alongside working with SQL Server, MySQL, and PostgreSQL databases, I have experience with Git/GitHub, REST API development, socket programming (TCP/UDP), and Linux (Ubuntu). I have worked on e-commerce platforms, healthcare and blood donation management systems, client-server architecture-based applications, and AI-powered projects.

In web development, I also have knowledge of SEO-compliant web structure, semantic HTML, meta tags, SEO-friendly URL structures, responsive design, web performance, sitemaps, robots.txt, and basic web security. I have carried out work on API integrations, user interfaces, database structures, and frontend-backend communication.

I have also carried out academic and applied work in data mining, machine learning, and data analysis. As a computer engineer focused on analytical thinking, problem-solving, and continuous learning, I continue to develop myself in Full Stack Development, Backend Development, Web Technologies, and Artificial Intelligence, and to contribute to real-world projects.

LANGUAGES

- Arabic (Native)
- Turkish (Good level - B2)
- English (Basic level - A2)

INTERNSHIP EXPERIENCE

Web Development Intern – Speed Of Light Company (August 2024)

- Learned version control processes using Git and GitHub; managed project tracking through repository creation, commit, push, and pull operations.
- Worked on product matching, stock tracking, and order management processes on marketplaces such as Trendyol, Hepsiburada, N11, Çiçeksepeti, and PTT AVM.
- Developed responsive web interfaces using HTML5 and CSS3 (Flexbox, Grid, animations, media queries).
- Built interactive and animated UI components using jQuery, Owl Carousel, SweetAlert2, Tippy.js, and Micron.js libraries.
- Developed backend operations with PHP and object-oriented programming (OOP); established MySQL database connections using mysqli and PDO.
- Developed a responsive website with navbar, card, form, and button components using Bootstrap.

Software Development Intern – Miraz Teknoloji (August 2025)

- Developed user interfaces with React; built and deployed a dream-journal application with ChatGPT API integration on Vercel.
- Practiced mobile app development by setting up the React Native environment and using core components (Text, Image, Button, FlatList, Switch, etc.) with TypeScript.
- Carried out work on image processing.
- Developed a project on content and site management with WordPress (articles, pages, users, Elementor, header/footer, menus).

PROJECTS

Blood Donation Management System — Solo Graduation Project

Developed to solve critical problems encountered in blood donation processes, this is an AI-powered healthcare management and decision-support platform. The project aims to solve problems such as insufficient blood stocks, the inability to reach suitable donors in time, donor management, coordination of emergency blood needs, communication between hospitals and blood centers, and time loss in operational processes.

The main goal of the project is to bring donors, healthcare institutions, blood centers, and system administrators together on a central digital platform, enabling blood donation processes to be managed faster, more efficiently, and in a more data-driven way. The system focuses not only on tracking current blood stocks but also on predicting future blood-type shortages, identifying suitable donors, managing emergencies, and providing data-driven recommendations to administrators.

The platform consists of three main components: a React Native-based mobile application, a Next.js 14-based web admin panel, and a FastAPI-based asynchronous backend service. The mobile application was developed using React Native and TypeScript. Through the mobile application, operations such as user registration, donor profile management, blood donation history, information about donation eligibility, donation-related notifications, and emergency blood requests can be carried out.

The web admin panel was developed using Next.js 14, TypeScript, and Tailwind CSS technologies. Through the admin panel, system administrators can track blood stocks, manage donors and blood requests, review hospital-related information, analyze system data, and coordinate blood donation operations.

The backend infrastructure of the system was developed using Python 3.11+ and FastAPI with AsyncIO support. FastAPI provides a high-performance API layer between the mobile application, the web panel, and the database. Thanks to the asynchronous architecture, a large number of simultaneous requests are aimed to be managed more efficiently.

PostgreSQL 15+ is used for data management. The system consists of more than 60 related database tables in order to manage different types of data such as users, donors, blood types, donation records, blood stocks, hospitals, emergency blood requests, notifications, location information, appointments, and analysis data.

One of the most important features of the project is its artificial intelligence and data mining layer. The system was not designed merely as a management system based on traditional database queries; machine learning and data mining algorithms were also incorporated into the system to build decision-support mechanisms.

In this context, machine learning algorithms such as Random Forest, LightGBM, and XGBoost are used for prediction and classification operations (appointment no-show risk and donor response likelihood), while the Apriori algorithm is used to discover meaningful relationships and associations within blood donation data. Blood stock levels are forecast 7–30 days ahead with an ARIMA/Prophet/LSTM ensemble, and a hybrid ranker that combines rule-based scoring, AHP/TOPSIS, and a learned score prioritizes which donor to contact first. A Graph Neural Network (GraphSAGE) research prototype for donor-request matching was developed and achieved 78% accuracy in laboratory testing. With an inference time of approximately 2.3 seconds, it is currently not included in the live matching flow. The project does not have real users yet, so all models were trained and evaluated on synthetic donor, appointment, and blood-stock datasets that I generated myself.

The AI layer can provide decision support in various scenarios. For example, by analyzing historical blood donation data, current blood stocks, and demand trends, the system can help predict future blood-type shortages. At the same time, by analyzing donors' past donation information and other data in the system, it can help identify donors who may be suitable for a specific blood need.

Another important component of the system is blood stock and shortage management. Blood stocks are not kept merely as static data; current stock quantity, blood type, past donations, and demand information are evaluated together. Through the admin panel, administrators can track current blood types, stock levels, blood types reaching critical levels, new donations, and emergency needs. This aims to reduce the need for manually collecting information from different sources and the associated time loss.

The system also has an emergency blood need coordination mechanism. Emergency blood requests can be created and tracked through the system. Notifications can be sent to suitable donors to enable faster communication in emergency situations. Location information is also used to evaluate the relationships between donors, hospitals, and blood needs, aiming to make the coordination process more efficient.

In terms of software architecture, the project is built on a multi-layered, service-oriented architecture. The mobile application and the web admin panel communicate through the FastAPI backend service. The backend layer manages authentication, authorization, business logic, database operations, AI services, notifications, and other processes within the system. PostgreSQL forms the system's persistent data storage layer.

The project was also developed with security, scalability, sustainability, and modularity principles in mind. The permissions of different user roles within the system are controlled, and the backend contains modular structures such as user management, donor management, blood stocks, requests, hospitals, notifications, analytics, and AI services. This structure makes it easier to extend the system with new features in later stages.

In conclusion, the Blood Donation Management System was designed not as a classic CRUD application that merely keeps track of blood stocks, but as an AI-powered decision-making and coordination platform. It aims to apply mobile application technologies, modern web

development, asynchronous backend services, relational database management, machine learning, data mining, and graph-based learning (as a research prototype) to a real-world healthcare-focused problem.

The project combines many fields of computer engineering — such as Artificial Intelligence, Machine Learning, Data Mining, Graph Neural Networks (research prototype), Mobile App Development, Web Development, Backend Engineering, Database Management, and Software Architecture — within a single system. The ultimate goal is to build a scalable digital healthcare infrastructure that can contribute to managing blood stocks more effectively, detecting potential blood shortages earlier, reaching suitable donors faster, and responding to emergency blood needs more effectively.

Note: To protect intellectual property rights, the project's source code has not been made publicly available on GitHub; further details can be provided during the interview process upon request. The project is currently being prepared for a TÜBİTAK application.

Disaster Management Decision Support System (DSS) – Group Project

Technologies: Laravel, FastAPI, Python, MySQL, JavaScript, Leaflet, ApexCharts, REST API

- As a team, we developed a data-driven Decision Support System that helps determine which team should be directed to which region and at which priority level by analyzing needs in disaster areas.
- Developed REST APIs for data analysis and decision-support operations using FastAPI and Python.
- Created a priority-based team assignment mechanism that selects the nearest suitable available team by Haversine distance, with fallback tiers, based on regional needs and disaster conditions.
- Implemented a transparent, rule-based scoring: a weighted need score per region (water, food, tents, medicine) mapped to four priority levels, with simple rules suggesting team types.
- The dashboard and Leaflet map visualizations of critical disaster areas consume these services over REST.
- Provided REST API integration between the Laravel web application and the FastAPI backend.
- Designed a system that supports faster, data-driven decision-making in disaster response.

FitAI – AI-Powered SaaS Layer for Online Fashion Sellers (Group Project)

A multi-tenant, AI-powered platform concept that helps online fashion sellers reduce returns caused by incorrect size and product selection. It provides a fit-compatibility score between 0–100, a return-risk estimate, and advice by analyzing the shopper's body type (currently selected manually) together with the product's cut and fabric characteristics; Trendyol and Hepsiburada product links can be analyzed automatically. Sellers get an admin panel with store, product, review, and commission management and an NLP breakdown of customer reviews (sentiment and themes, using a pre-trained Turkish BERT model). The project was developed in 2025 by a 7-person team split into web (3), AI (2), and mobile (2) sub-teams. On the web team I was responsible for UI/UX: I drafted the design plan, designed the web interfaces (login/register, dashboard, stores, products, reviews, and analytics screens), and integrated the AI outputs into the frontend. The web panel uses ASP.NET Core (ABP Framework), Razor Pages, and EF Core; the AI services use Python, FastAPI, Scikit-learn, and Random Forest. The backend and the Flutter mobile app were built by teammates.

Opale Store – E-Commerce Platform (Group Project)

A comprehensive e-commerce platform developed for the sale of luxury fashion and cosmetic products. Core e-commerce modules such as user registration and login, product and category management, shopping cart, secure payment, order tracking, and an admin panel were developed in the project. The aim was to provide a secure and user-friendly shopping experience. Laravel, MySQL, HTML, CSS, and JavaScript technologies were used during development.

Not Takip Sistemi (Grade Tracking System) – Solo Project

A two-sided web application for managing and publishing academic information: a public site where visitors can browse paginated professor and student directories, read professor detail pages, and see approved course reviews, and an admin panel behind email/password login. The admin panel provides full CRUD for students, professors, courses, grades, comments, and site contact information, photo uploads for professors and students, a dashboard summarizing student, professor, and pending-comment counts, and a comment-moderation flow in which a review becomes public only after admin approval. I designed and built the whole system on my own, from the SQL Server schema (nine related tables) and the Entity Framework data layer to every controller and view. ASP.NET MVC 5 (.NET Framework 4.7.2), Entity Framework 6 (Database-First), SQL Server, Razor, Bootstrap 5, jQuery, and DataTables were used during development.